

Analisi – Servizio Gestione Licenze

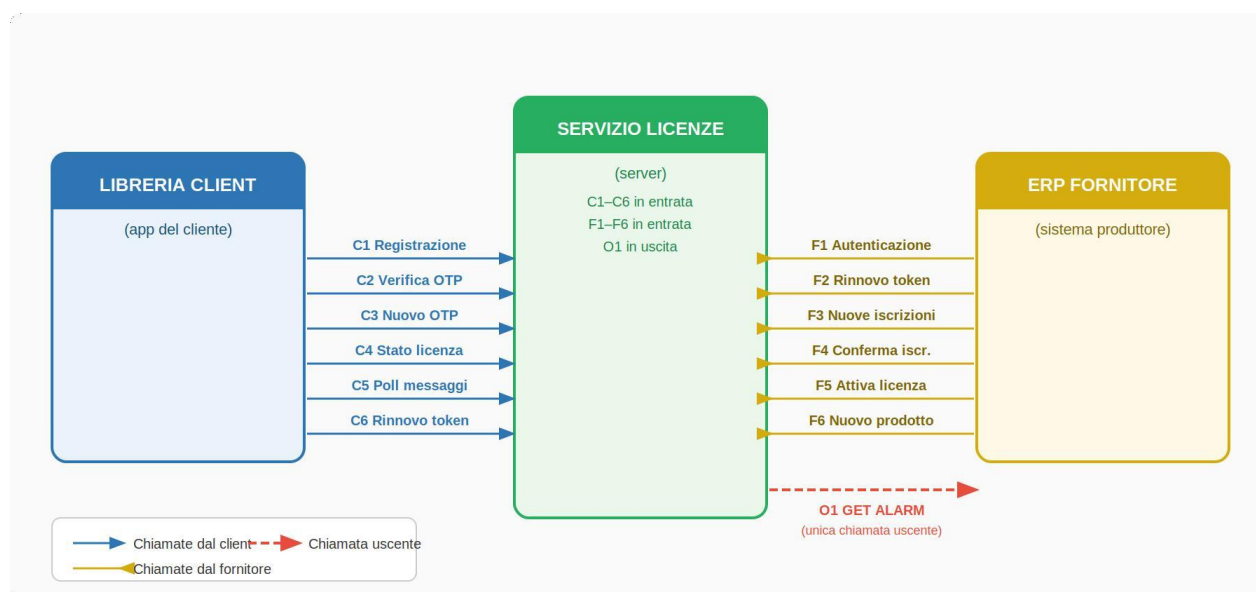
v. 1.0 | Pavan Andrea | Pavan Desirée

1. Panoramica generale

Il Servizio Gestione Licenze è un sistema che si posiziona tra la **libreria client** e il **sistema ERP del produttore**. Il servizio non inizia mai comunicazioni verso i clienti o il produttore di propria iniziativa, ad eccezione di un'unica notifica uscente verso l'ERP (il GET ALARM) e dei messaggi email/in-app schedulati.

Il suo scopo principale è **gestire le licenze software** dalla prima registrazione del cliente, all'attivazione della Trial Demo, fino alla Licenza Provvisoria, al rinnovo e alla scadenza. Ogni operazione transita attraverso questo servizio, che aggiorna costantemente lo stato delle licenze.

Architettura a tre livelli



Legenda:

- Freccie blu: chiamate libreria client (C1–C6);
- Freccie gialle: chiamate del produttore (F1–F8);
- Freccia rossa tratteggiata: GET ALARM (O1), unica chiamata uscente.

Punti chiave

- Il servizio è passivo: risponde alle chiamate, non le inizia (ad eccezione di O1 e messaggi schedulati)
- La P.IVA/VAT viene validata al momento della registrazione tramite il servizio VIES (sistema UE, copre tutti i paesi UE)
- Ogni cliente è identificato da una license key univoca generata con salt random al momento dell'attivazione

- Ad ogni attivazione o rinnovo viene generato un file crittografato con chiave di decrittazione per il funzionamento offline
- La frequenza delle chiamate C4 e C5 è configurabile dal produttore tramite `check_frequency_hours`
- I template dei messaggi sono gestiti in DB nella tabella `email_templates` con `event_code` e placeholder sostituibili
- Sono supportati quattro tipi di licenza: `trial`, `monthly`, `annual`, `provisional`
- Il sistema è multilingua (italiano e inglese) e predisposto per futura gestione multi-produttore

2. Descrizione dettagliata del funzionamento

Di seguito viene descritto il funzionamento completo del servizio seguendo l'ordine naturale dei processi: dalla configurazione iniziale del produttore, alla registrazione del cliente, fino agli scenari di scadenza, Licenza Provvisoria e casi eccezionali.

2.1 Configurazione iniziale del produttore

Prima che qualsiasi cliente possa registrarsi, il produttore deve autenticarsi sul servizio e registrare i propri prodotti. Questa fase avviene per ogni nuovo prodotto messo in commercio.

F1 – Autenticazione produttore

POST /api/vendor/auth/login

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Autenticazione del fornitore tramite API key statica. L'API key viene verificata tramite bcrypt (rounds=12) sull'hash salvato in DB. Emette JWT RS256 (TTL 60s) e refresh token (TTL 1h) per le chiamate successive. Rate limited su IP.

Header: nessuno

Request body:

```
{
  "api_key": "vk_chiave_statica_del_fornitore" // API key assegnata al
  fornitore - obbligatorio
}
```

Tabelle DB coinvolte:

vendors (lettura hash), vendor_tokens (inserimento), rate_limits (lettura/aggiornamento)

Controlli:

- api_key corrisponde all'hash bcrypt in vendors.api_key_hash
- API key non revocata (api_key_revoked_at IS NULL)
- Rate limiting: max 10 richieste/ora per IP sorgente

Risposte:

```
// 200 - Autenticazione OK
{
  "jwt": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "jwt_expires_in_seconds": 60,
  "refresh_token": "rt_vendor_abcl23",
  "refresh_token_expires_in_seconds": 3600,
  "vendor_id": 1
}

// 401 - API key non valida
{
  "error_code": "INVALID_API_KEY",
  "message": "API key non valida",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - API key revocata (dopo F9)
{
  "error_code": "API_KEY_REVOKED",
  "message": "API key revocata. Utilizzare la nuova chiave generata con F9.",
}
```

```

    "timestamp": "2026-06-09T10:00:00Z",
    "request_id": "uuid"
  }

  // 429 - Troppe richieste
  {
    "error_code": "RATE_LIMIT_EXCEEDED",
    "message": "Troppi tentativi. Riprovare tra 1 ora.",
    "details": { "retry_after_seconds": 3600 },
    "timestamp": "2026-06-09T10:00:00Z",
    "request_id": "uuid"
  }

```

Il sistema ERP del produttore si autentica inviando la propria API key statica. Il servizio verifica la chiave nella tabella **vendors** e restituisce un JWT di breve durata e un refresh token valido un'ora.

Tabella coinvolta: vendors

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del produttore
name	VARCHAR (255)	Nome del produttore
api_key	VARCHAR (255)	Chiave API statica usata per l'autenticazione (hash)
erp_alarm_url	VARCHAR (500)	URL dell'endpoint GET ALARM dell'ERP del produttore
check_frequency_hours	INT	Frequenza in ore con cui la libreria client deve chiamare C4 e C5 — configurata dal produttore
created_at	TIMESTAMP	Data e ora di inserimento del record

Tabella coinvolta: vendor_tokens

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
vendor_id	INT	Riferimento al produttore
refresh_token	VARCHAR (512)	Refresh token (hash) — ruotato ad ogni utilizzo
expires_at	TIMESTAMP	Scadenza del refresh token (1 ora)
revoked	BOOL	TRUE se revocato anticipatamente
created_at	TIMESTAMP	Data e ora di emissione

F2 – Rinnovo token produttore

POST /api/vendor/token/refresh

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Rinnova il JWT del fornitore tramite refresh token (rotation). Stessa logica di C6 applicata al vendor.

Header: nessuno

Request body:

```
{
  "refresh_token": "rt_vendor_abc123"    // Refresh token ricevuto da F1 o F2
precedente - obbligatorio
}
```

Tabelle DB coinvolte:

vendor_tokens (lettura, aggiornamento rotation)

Controlli:

- refresh_token valido e non scaduto (TTL 1h)
- Non già usato (monouso per rotation)

Risposte:

```
// 200 - Nuovi token emessi
{
  "jwt": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "jwt_expires_in_seconds": 60,
  "refresh_token": "rt_vendor_nuovo_ruotato",    // Il vecchio è invalidato
  "refresh_token_expires_in_seconds": 3600
}

// 401 - Refresh token non valido o scaduto
{
  "error_code": "INVALID_REFRESH_TOKEN",
  "message": "Refresh token non valido o scaduto. Eseguire nuovamente F1.",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

Quando il JWT scade, il produttore usa il refresh token per ottenerne uno nuovo. Il refresh token viene ruotato ad ogni utilizzo (token rotation).

F6 – Registrazione nuovo prodotto

POST /api/vendor/products

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Registra un nuovo prodotto nel sistema. Genera la product_key univoca da includere nella libreria client distribuita ai clienti del fornitore.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body:

```
{
  "product_name": "MioSoftware Pro", // Nome del prodotto -
  "trial_duration_days": 30,          // Durata trial in giorni -
  "trial_max_users": 1,               // Max utenti durante la
  "trial_modules": ["modulo_a"],      // Moduli inclusi nella
  "license_check_frequency_days": 7   // Frequenza check C4 in
}
```

Tabelle DB coinvolte:

products (inserimento), modules (inserimento/associazione)

Controlli:

- JWT vendor valido
- product_name non vuoto
- trial_duration_days > 0
- license_check_frequency_days > 0

Risposte:

```
// 201 - Prodotto registrato
{
  "product_key": "PK-A1B2C3D4", // Chiave da includere nella libreria
  "product_name": "MioSoftware Pro",
  "trial_duration_days": 30,
  "trial_max_users": 1,
  "trial_modules": ["modulo_a"],
  "license_check_frequency_days": 7
}

// 400 - Campo obbligatorio mancante
{
  "error_code": "MISSING_REQUIRED_FIELD",
  "message": "Il campo product_name è obbligatorio",
  "details": { "field": "product_name" },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - JWT non valido
```

```
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

Il produttore registra un nuovo prodotto comunicando la sua `product_key` e il nome. La chiave viene inserita nella tabella **products** e da quel momento è valida per le registrazioni dei clienti che acquisteranno quell'applicazione.

Tabella coinvolta: products

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del prodotto
product_key	VARCHAR(100)	Chiave univoca del prodotto, inclusa nella libreria client
name	VARCHAR(255)	Nome leggibile del prodotto
created_at	TIMESTAMP	Data e ora di inserimento del record

2.2 Registrazione del cliente

Quando un cliente installa per la prima volta un'applicazione del produttore, la libreria client avvia automaticamente il processo di registrazione in due fasi: invio dei dati e verifica OTP.

C1 – Registrazione cliente

POST /api/client/register

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Prima iscrizione del cliente al servizio. Verifica che la chiave prodotto esista e che il cliente non sia già registrato. Se è la prima registrazione, salva i dati in stato pending e invia un codice OTP via email per la verifica dell'indirizzo. Se il cliente è già registrato con licenza attiva, restituisce direttamente i dati della licenza (caso riapertura app — nessun OTP necessario).

Header: nessuno (endpoint pubblico — rate limited per IP)

Request body:

```
{
  "product_key": "string", // Chiave prodotto inclusa nella libreria client
  - obbligatorio
  "vat_number": "string", // P.IVA o VAT number (formato libero per clienti
  esteri) - obbligatorio
  "company_name": "string", // Ragione sociale - obbligatorio
  "country": "string", // Codice paese ISO 3166-1 alpha-2 (es. IT, DE,
  FR) - obbligatorio
  "contact_email": "string", // Email per notifiche, deve essere valida (RFC
  5322) - obbligatorio
  "contact_phone": "string", // Telefono - opzionale
  "referent_name": "string" // Nome e cognome del referente - opzionale
}
```

Tabelle DB coinvolte:

products (lettura), clients (lettura/inserimento), otp_codes (inserimento), rate_limits (lettura/aggiornamento)

Controlli:

- product_key esiste in products
- contact_email in formato valido
- vat_number + product_key già presenti con licenza attiva → risposta 200 (nessun OTP)
- vat_number + product_key già presenti con trial scaduta e non rinnovata → risposta 409
- Rate limiting: max 5 richieste/ora per IP sorgente

Risposte:

```
// 201 - Prima registrazione, OTP inviato via email
{
  "status": "pending_verification",
  "client_id": 42, // ID da passare a C2 e C3
  "message": "Codice OTP inviato all'indirizzo email fornito"
}

// 200 - Già registrato con licenza attiva (riapertura app)
{
  "status": "already_registered",
  "license_key": "lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h",
  "license_type": "trial|normal",
  "expires_at": "2026-12-31T23:59:59Z",
  "modules": ["modulo_a", "modulo_b"]
}

// 400 - Campo obbligatorio mancante
{
  "error_code": "MISSING_REQUIRED_FIELD",
  "message": "Il campo product_key è obbligatorio",
  "details": { "field": "product_key" },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 400 - Email non valida
{
  "error_code": "INVALID_EMAIL_FORMAT",
  "message": "Il formato dell'email non è valido",
  "details": { "field": "contact_email" },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 404 - Chiave prodotto non trovata
{
  "error_code": "INVALID_PRODUCT_KEY",
  "message": "Chiave prodotto non trovata",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 409 - Trial già utilizzata per questo prodotto
{
  "error_code": "TRIAL_ALREADY_USED",
  "message": "La trial per questo prodotto è già stata utilizzata. Contattare il fornitore per attivare una licenza.",
  "timestamp": "2026-06-09T10:00:00Z",
```



```

    "request_id": "uuid"
  }

  // 429 - Troppe richieste
  {
    "error_code": "RATE_LIMIT_EXCEEDED",
    "message": "Troppi tentativi. Riprovare tra 1 ora.",
    "details": { "retry_after_seconds": 3600 },
    "timestamp": "2026-06-09T10:00:00Z",
    "request_id": "uuid"
  }

```

La libreria client invia i dati anagrafici: product_key, P.IVA o VAT number, ragione sociale, paese, email di contatto, lingua preferita e opzionalmente telefono e referente.

Il servizio esegue una **validazione formale** della P.IVA/VAT (formato corretto per il paese indicato) e una **validazione sostanziale tramite VIES**, il sistema UE che verifica che la P.IVA esista ed sia attiva nel registro europeo. Il risultato viene salvato nel campo vat_verified. Verifica poi che la coppia vat_number + country non sia già associata a quella product_key: se lo fosse, il tentativo viene bloccato (vedi sezione 2.8).

Se i dati sono validi, il cliente viene salvato nella tabella **clients** con stato pending e viene inviata un'email OTP.

Tabella coinvolta: clients

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del cliente
vat_number	VARCHAR(30)	P.IVA o VAT number — formato libero, validato tramite VIES
country	VARCHAR(2)	Codice paese ISO (es. IT, DE, FR) — usato per validazione VIES
company_name	VARCHAR(255)	Ragione sociale dell'azienda cliente
contact_email	VARCHAR(255)	Email per notifiche e OTP
language	VARCHAR(2)	Lingua preferita — valori: it, en
contact_phone	VARCHAR(30)	Telefono — opzionale
referent_name	VARCHAR(255)	Nome e cognome del referente — opzionale
vat_verified	BOOL	TRUE se la P.IVA è stata verificata con successo tramite VIES
status	ENUM	Stato: pending (attesa OTP) oppure active
created_at	TIMESTAMP	Data e ora di inserimento del record

Tabella coinvolta: otp_codes

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
client_id	INT	Riferimento al cliente in clients

code	VARCHAR(10)	Codice OTP generato (es. 6 cifre)
expires_at	TIMESTAMP	Scadenza del codice (es. 15 minuti)
used_at	TIMESTAMP	Data utilizzo — NULL se non ancora usato
created_at	TIMESTAMP	Data e ora di generazione

In questa fase viene inviata la seguente email al cliente:

Email OTP → Cliente (event_code: REGISTRATION_OTP)

Oggetto: Verifica il tuo indirizzo email – {product_name}

Gentile {company_name},

abbiamo ricevuto la sua richiesta di registrazione a {product_name}.

Per completare la registrazione e attivare la Trial Demo, la invitiamo a inserire il seguente codice di verifica nell'applicazione:

Codice OTP: {otp_code}

Il codice è valido per {otp_expiry_minutes} minuti. Se non ha effettuato questa richiesta, può ignorare questa email.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

C3 – Nuovo OTP (caso: OTP scaduto o non ricevuto)

POST /api/client/resend-otp

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Genera e invia un nuovo codice OTP quando quello precedente è scaduto. Invalida l'OTP precedente e ne crea uno nuovo con un nuovo TTL.

Header: nessuno (endpoint pubblico — rate limited per client)

Request body:

```
{
  "client_id": 42    // ID cliente in stato pending — obbligatorio
}
```

Tabelle DB coinvolte:

clients (lettura), otp_codes (aggiornamento), rate_limits (lettura/aggiornamento)

Controlli:

- client_id esiste con status = pending
- Rate limiting: max 3 richieste/ora per client_id

Risposte:

```
// 200 - Nuovo OTP inviato
{
  "status": "otp_sent",
  "message": "Nuovo codice OTP inviato all'indirizzo email registrato"
}

// 404 - client_id non trovato o non in stato pending
{
  "error_code": "CLIENT_NOT_FOUND",
  "message": "ID cliente non trovato o non in attesa di verifica",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 429 - Troppe richieste
{
  "error_code": "RATE_LIMIT_EXCEEDED",
  "message": "Troppi tentativi. Riprovare tra 1 ora.",
  "details": { "retry_after_seconds": 3600 },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

Se il cliente non riceve l'OTP o se scade prima che venga inserito, la libreria può richiederne uno nuovo. Il servizio invalida il codice precedente nella tabella `otp_codes` e ne genera uno nuovo, inviando nuovamente l'email di verifica.

C2 – Verifica OTP

POST /api/client/verify-otp

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Verifica il codice OTP ricevuto via email. Se valido: attiva il cliente (status → active), crea la licenza trial, genera la `license_key` univoca (HMAC-SHA256 + salt random), il JWT RS256 (TTL 60s), il refresh token (TTL 1h) e l'`offline_token` crittografato (AES-256-GCM). Imposta `vendor_synced = false` — la notifica O1 all'ERP è delegata al job schedulato `NEW_REGISTRATION` (non avviene in real-time, vedi sezione 12). Idempotente: se C2 viene chiamata una seconda volta con gli stessi dati, restituisce gli stessi token senza creare duplicati.

Header: nessuno (endpoint pubblico)

Request body:

```
{
  "client_id": 42,           // ID cliente ricevuto dalla risposta di C1 -
  "otp_code": "482931"      // Codice OTP a 6 cifre ricevuto via email -
}
```

Tabelle DB coinvolte:

`clients` (aggiornamento), `otp_codes` (lettura/eliminazione), `otp_attempts` (lettura/aggiornamento), `licenses` (inserimento), `client_tokens` (inserimento)

Controlli:

- `client_id` esiste in `clients` con status = pending (se status = active → idempotente, restituisce dati esistenti)
- `otp_code` corrisponde all'hash SHA256 salvato in `otp_codes`
- OTP non scaduto (TTL configurabile per prodotto)
- Max 3 tentativi falliti consecutivi → lockout 30 minuti (tabella `otp_attempts`)

Azioni in caso di successo:

3. `clients.status` → active
4. Genera `license_key` con HMAC-SHA256 + salt random
5. Crea licenza trial in `licenses` (durata e moduli da `products`)
6. Genera `offline_token` con AES-256-GCM
7. Genera JWT RS256 (TTL 60s) + refresh token (TTL 1h)
8. Imposta `licenses.vendor_synced = false`

Risposte:

```
// 201 - OTP verificato, trial attivata
{
  "status": "trial_activated",
  "license_key": "lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h", // Salvare nella
  "license_type": "trial",
}
```

```

    "expires_at": "2026-12-31T23:59:59Z",
    "modules": ["modulo_a", "modulo_b"],
    "jwt": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
    "jwt_expires_in_seconds": 60,
    "refresh_token": "rt_c3d4e5f6g7h8i9j0k1l2m3n4", // Salvare per C6
    "offline_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...", // Salvare
    localmente
    "offline_token_expires_at": "2026-06-18T15:30:00Z"
  }

// 401 - OTP errato
{
  "error_code": "INVALID_OTP",
  "message": "Codice OTP non valido",
  "details": { "attempts_remaining": 2 },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - OTP scaduto
{
  "error_code": "OTP_EXPIRED",
  "message": "Il codice OTP è scaduto. Richiedere un nuovo codice tramite C3.",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 403 - Troppi tentativi falliti (lockout)
{
  "error_code": "OTP_MAX_ATTEMPTS",
  "message": "Troppi tentativi falliti. Accesso bloccato per 30 minuti.",
  "details": { "retry_after_seconds": 1800 },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 404 - client_id non trovato
{
  "error_code": "CLIENT_NOT_FOUND",
  "message": "ID cliente non trovato",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

```

Il cliente inserisce il codice OTP nell'applicazione e la libreria lo invia al servizio. Se valido e non scaduto, il servizio:

- Attiva il cliente (status → active)
- Genera la license_key univoca con salt random
- Crea la licenza trial nella tabella licenses
- Genera il file crittografato per uso offline (license_file) e la chiave di decrittazione (license_decrypt_key) con stessa scadenza della licenza
- Emette JWT + refresh token, incluso il parametro check_frequency_hours
- Triggera il GET ALARM verso l'ERP del produttore

Tabella coinvolta: licenses

Campo	Tipo	Descrizione
id	INT	Identificativo univoco della licenza
client_id	INT	Riferimento al cliente in clients

product_id	INT	Riferimento al prodotto in products
license_key	VARCHAR(255)	Chiave univoca generata con salt random — identificativo usato dalla libreria client
license_type	ENUM	Tipo: trial, monthly, annual, provisional
status	ENUM	Stato: active, expired, suspended
max_users	INT	Numero massimo utenti — NULL per trial se non definito
starts_at	TIMESTAMP	Inizio validità
expires_at	TIMESTAMP	Scadenza
activated_at	TIMESTAMP	Data di attivazione
deactivated_at	TIMESTAMP	Data disattivazione — NULL se ancora attiva
vendor_synced	BOOL	FALSE finché il produttore non conferma via F4
license_file	TEXT	File crittografato (Base64) con dati licenza per uso offline — rigenerato ad ogni attivazione/rinnovo
license_decrypt_key	VARCHAR(512)	Chiave di decrittazione — stessa scadenza della licenza, rigenerata ad ogni attivazione/rinnovo
created_at	TIMESTAMP	Data di inserimento del record

Tabella coinvolta: client_tokens

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
client_id	INT	Riferimento al cliente in clients
refresh_token	VARCHAR(512)	Refresh token (hash) — ruotato ad ogni utilizzo
expires_at	TIMESTAMP	Scadenza del refresh token (1 ora)
revoked	BOOL	TRUE se revocato anticipatamente
created_at	TIMESTAMP	Data e ora di emissione

In questa fase vengono inviati i seguenti messaggi:

Email di benvenuto → Cliente (event_code: WELCOME)

Oggetto: Benvenuto in {product_name} – Attivazione Trial Demo

Gentile {company_name},

siamo lieti di comunicarle che la Trial Demo di {product_name} è stata attivata con successo.

Il periodo di prova le permetterà di esplorare le funzionalità del prodotto e valutarne i benefici per la sua azienda. La Trial Demo sarà disponibile fino al {expires_at}.

Al termine del periodo di prova, potrà procedere con l'acquisto di una licenza mensile o annuale per continuare ad usufruire del servizio senza interruzioni.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Messaggio in-app → Cliente (event_code: WELCOME)

Titolo: Benvenuto in {product_name}

Testo: La Trial Demo è stata attivata con successo. Ha a disposizione il periodo di prova fino al {expires_at} per esplorare tutte le funzionalità del prodotto.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Email nuova registrazione → Produttore

Oggetto: Nuova registrazione cliente – {company_name}

Gentile team,

si comunica che un nuovo cliente si è registrato al servizio.

Dettagli registrazione:

Azienda: {company_name}
Prodotto: {product_name}
Tipo licenza attivata: Trial Demo
Scadenza Trial: {expires_at}

Cordiali saluti,
Il sistema di gestione licenze

8.1 Funzionamento ordinario della licenza e modalità offline

Una volta registrato, il client usa la `license_key` e il JWT per tutte le comunicazioni. La frequenza delle chiamate è definita da `check_frequency_hours` restituito in C2. Quando il client è **offline**, utilizza il file crittografato salvato localmente per verificare la validità della licenza senza chiamare il server. Quando torna online, C4 ha sempre la precedenza sul file locale.

C4 – Verifica stato licenza

GET /api/client/license/status

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Check periodico dello stato della licenza. Restituisce stato corrente, moduli attivi e rinnova l'offline_token. La frequenza del check è configurabile per prodotto (`license_check_frequency_days`). Se la licenza risulta scaduta, C4 lo comunica al client ma non aggiorna il DB né chiama O1 — queste operazioni sono delegate al job schedulato `LICENSE_EXPIRED` (sezione 12).

Header:

```
Authorization: Bearer <jwt>  
X-License-Key: lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h
```

Query parameters: nessuno

Tabelle DB coinvolte:

`licenses` (lettura), `client_tokens` (verifica JWT), `products` (lettura frequenza), `client_activity_logs` (aggiornamento `last_c5_at`)

Controlli:

- JWT valido e non scaduto (RS256, TTL 60s) — se scaduto → 401, il client chiama C6
- `license_key` corrisponde al client nel payload JWT

Risposte:

```
// 200 - Licenza attiva  
{  
  "status": "active",  
  "license_type": "trial|normal",  
  "expires_at": "2026-12-31T23:59:59Z",  
  "modules": ["modulo_a", "modulo_b"],  
  "offline_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...", // Token
```



```

aggiornato, da sovrascrivere
  "offline_token_expires_at": "2026-06-18T15:30:00Z",
  "next_check_in_days": 7    // Da products.license_check_frequency_days
}

// 200 - Licenza scaduta (01 e aggiornamento DB delegati al job LICENSE_EXPIRED)
{
  "status": "expired",
  "license_type": "trial|normal",
  "expired_at": "2026-06-01T00:00:00Z"
}

// 200 - Licenza revocata
{
  "status": "revoked",
  "revoked_at": "2026-06-05T12:00:00Z"
}

// 401 - JWT non valido o scaduto
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto. Utilizzare C6 per rinnovarlo.",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 404 - Licenza non trovata
{
  "error_code": "LICENSE_NOT_FOUND",
  "message": "Licenza non trovata per la license_key fornita",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

```

La libreria client chiama questo endpoint periodicamente per verificare lo stato della licenza. Il servizio controlla JWT e license_key, aggiorna lo stato se scaduta (active → expired) e restituisce tipo, scadenza, moduli abilitati e massimo utenti.

Tabella coinvolta: modules

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del modulo
name	VARCHAR(100)	Nome identificativo del modulo (es. modulo_contabilita)
description	VARCHAR(255)	Descrizione leggibile — opzionale
created_at	TIMESTAMP	Data di inserimento del record

Tabella coinvolta: license_modules

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
license_id	INT	Riferimento alla licenza in licenses
module_id	INT	Riferimento al modulo in modules

C5 – Poll messaggi in-app
GET /api/client/messages

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Poll periodico per recuperare i messaggi in-app in coda (avvisi di scadenza, notifiche di rinnovo, comunicazioni del fornitore). Aggiorna last_seen_at nel log attività e segna i messaggi come consegnati (delivered_at).

Header:

```
Authorization: Bearer <jwt>
X-License-Key: lk...
```

Query parameters: nessuno

Tabelle DB coinvolte:

messages (lettura/aggiornamento), client_activity_logs (aggiornamento)

Controlli:

- JWT valido

Risposte:

```
// 200 - Lista messaggi (array vuoto se non ci sono messaggi in coda)
{
  "messages": [
    {
      "id": 101,
      "template_key": "SCADENZA_IMMINENTE",
      "content": "La tua licenza scade tra 7 giorni. Contatta il fornitore per il rinnovo.",
      "created_at": "2026-06-02T10:00:00Z"
    }
  ]
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto. Utilizzare C6 per rinnovarlo.",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

La libreria client fa polling periodico per ricevere messaggi in-app in coda. Il servizio restituisce i messaggi con delivered_at NULL, li marca come consegnati, aggiorna last_seen_at in client_activity_logs e genera i messaggi dai template in email_templates.

Tabella coinvolta: messages

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del messaggio
license_id	INT	Riferimento alla licenza — NULL se non legato a una licenza specifica
template_id	INT	Riferimento al template in email_templates
target	ENUM	Destinatario: client oppure vendor
channel	ENUM	Canale: email oppure in_app
type	ENUM	Tipo: banner, alert, info

language	VARCHAR (2)	Lingua: it oppure en
title	VARCHAR (255)	Titolo del messaggio
body	TEXT	Corpo con placeholder già sostituiti
cta_url	VARCHAR (500)	URL call-to-action — opzionale
delivered_at	TIMESTAMP	NULL finché non consegnato
created_at	TIMESTAMP	Data di creazione del messaggio

Tabella coinvolta: email_templates

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del template
event_code	VARCHAR (50)	Codice evento (es. WELCOME, TRIAL_EXPIRING, PROVISIONAL_ACTIVATED, ecc.)
channel	ENUM	Canale: email oppure in_app
target	ENUM	Destinatario: client oppure vendor
language	VARCHAR (2)	Lingua: it oppure en
subject	VARCHAR (255)	Oggetto email — NULL per in_app
title	VARCHAR (255)	Titolo in-app — NULL per email
body	TEXT	Corpo con placeholder (es. {company_name}, {expires_at}, {days_remaining}, ecc.)
created_at	TIMESTAMP	Data inserimento
updated_at	TIMESTAMP	Data ultima modifica

Tabella coinvolta: client_activity_logs

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
license_id	INT	Riferimento alla licenza in licenses
last_seen_at	TIMESTAMP	Data e ora dell'ultima chiamata C5 ricevuta
inactivity_notified_at	TIMESTAMP	Data invio email inattività al produttore — NULL se non ancora inviata

C6 – Rinnovo token cliente

POST /api/client/token/refresh

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Rinnova il JWT del client usando il refresh token. Il refresh token viene ruotato ad ogni utilizzo: il vecchio viene invalidato e ne viene emesso uno nuovo. Se il refresh token è scaduto (dopo 1h di

inattività), il client deve contattare il produttore per ottenere un nuovo accesso.

Header: nessuno

Request body:

```
{
  "refresh_token": "rt_c3d4e5f6g7h8i9j0k1l2" // Refresh token ricevuto da C2 o
  C6 precedente - obbligatorio
}
```

Tabelle DB coinvolte:

client_tokens (lettura, aggiornamento rotation)

Controlli:

- refresh_token esiste in client_tokens ed è ancora valido (TTL 1h)
- Non già usato in precedenza (ogni refresh token è monouso)

Risposte:

```
// 200 - Nuovi token emessi
{
  "jwt": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "jwt_expires_in_seconds": 60,
  "refresh_token": "rt_nuovo_ruotato_abc123", // Il vecchio è invalidato -
  salvare questo
  "refresh_token_expires_in_seconds": 3600
}

// 401 - Refresh token non valido o già usato
{
  "error_code": "INVALID_REFRESH_TOKEN",
  "message": "Refresh token non valido o già utilizzato",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - Refresh token scaduto (1h inattività)
{
  "error_code": "REFRESH_TOKEN_EXPIRED",
  "message": "Sessione scaduta per inattività. Il cliente deve contattare il
  produttore.",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

Il JWT del client ha durata molto breve. Quando scade, la libreria usa il refresh token per ottenerne uno nuovo. Il refresh token viene ruotato ad ogni utilizzo. Se anche il refresh token è scaduto (dopo 1 ora di inattività), il client dovrà contattare il produttore per ripristinare l'accesso.

8.2 Sincronizzazione con il produttore

Ogni volta che un cliente completa la registrazione, il servizio invia immediatamente un GET ALARM all'ERP del produttore.

01 – GET ALARM verso ERP produttore

GET {vendor_erp_url}/alarm

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo: Chiamata uscente dal Service Invoice verso l'ERP del fornitore. Notifica eventi rilevanti

(nuova registrazione, licenza in scadenza, licenza scaduta). Viene eseguita esclusivamente dai job schedulati del sistema eventi (sezione 12) — mai in risposta diretta a una chiamata API. L'URL di destinazione è configurato in `vendors.erp_alarm_url`.

Chiamata HTTP uscente (il Service Invoice chiama l'ERP):

```
GET {vendors.erp_alarm_url}/alarm
?alarm_code=NEW_REGISTRATION
&license_key=lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h
&vat_number=IT12345678901
&company_name=Acme+Srl
&product_key=PK-XYZ
&timestamp=2026-06-09T12:00:00Z
```

Valori possibili di `alarm_code`:

Valore	Descrizione	Job che lo attiva
NEW_REGISTRATION	Nuovo cliente registrato (vendor_synced = false)	NEW_REGISTRATION
LICENSE_EXPIRING	Licenza in scadenza entro soglia configurata	LICENSE_EXPIRING
LICENSE_EXPIRED	Licenza scaduta oggi	LICENSE_EXPIRED

Comportamento in base alla risposta dell'ERP:

```
HTTP 200 OK → alarm_logs: success = true
HTTP altro → alarm_logs: success = false, retry_count = 0
               Job ALARM_RETRY riproverà (max 3 tentativi)
               Al 3° fallimento → email GET_ALARM_FALLBACK al fornitore
                               alarm_logs: permanently_failed = true
```

Tabelle DB coinvolte: `alarm_logs` (inserimento/aggiornamento), `licenses` (aggiornamento `vendor_synced` se successo)

Unica chiamata uscente del server. Viene triggerata da: nuova registrazione, scadenza imminente, attivazione licenza provvisoria, conferma pagamento. I valori di `alarm_code` possibili sono `NEW_REGISTRATION`, `LICENSE_EXPIRING`, `LICENSE_EXPIRED`, `PROVISIONAL_ACTIVATED`, `PAYMENT_CONFIRMED`.

Tabella coinvolta: `alarm_logs`

Campo	Tipo	Descrizione
<code>id</code>	INT	Identificativo univoco del log
<code>alarm_code</code>	ENUM	Tipo di evento: <code>NEW_REGISTRATION</code> <code>LICENSE_EXPIRING</code> <code>LICENSE_EXPIRED</code> <code>PROVISIONAL_ACTIVATED</code> <code>PAYMENT_CONFIRMED</code>
<code>license_id</code>	INT	Riferimento alla licenza — NULL se non applicabile
<code>sent_at</code>	TIMESTAMP	Data e ora invio
<code>response_status</code>	INT	Codice HTTP restituito dall'ERP
<code>success</code>	BOOL	TRUE se l'ERP ha risposto 200

F3 – Recupero nuove iscrizioni

GET /api/vendor/registrations/new

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Restituisce la lista paginata delle nuove registrazioni clienti non ancora sincronizzate con l'ERP (vendor_synced = false). Il fornitore scarica i dati, li processa nel proprio sistema e poi chiama F4 per confermare la ricezione.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Query parameters:

```
?page=1          // Numero di pagina – default 1
&limit=50        // Risultati per pagina – default 50, massimo 100
```

Tabelle DB coinvolte:

clients (lettura), licenses (lettura, filtro vendor_synced = false), products (lettura)

Controlli:

- JWT vendor valido
- page e limit sono interi positivi; limit ≤ 100

Risposte:

```
// 200 - Lista registrazioni non sincronizzate
{
  "data": [
    {
      "registration_id": 42,          // ID da passare a F4 per la conferma
      "vat_number": "IT12345678901",
      "company_name": "Acme Srl",
      "country": "IT",
      "contact_email": "admin@acme.it",
      "contact_phone": "+39 02 1234567",
      "referent_name": "Mario Rossi",
      "product_key": "PK-XYZ",
      "license_type": "trial",
      "registered_at": "2026-06-09T10:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 50,
    "total": 1,
    "total_pages": 1
  }
}

// 400 - Parametri di paginazione non validi
{
  "error_code": "INVALID_PAGE_PARAMETER",
  "message": "I parametri page e limit devono essere interi positivi (limit max 100)",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
```

```
"message": "Token non valido o scaduto. Eseguire F1 o F2.",  
"timestamp": "2026-06-09T10:00:00Z",  
"request_id": "uuid"  
}
```

Ricevuto il GET ALARM, il produttore chiama questo endpoint per scaricare le nuove iscrizioni non ancora processate (`vendor_synced = false`). Il servizio restituisce tutti i dati del cliente e della licenza, inclusi `license_key` e `vat_verified`.

F4 – Conferma ricezione nuove iscrizioni

POST /api/vendor/registrations/confirm

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Conferma che l'ERP ha ricevuto e processato le registrazioni scaricate con F3. Imposta `vendor_synced = true` per gli ID confermati, che non verranno più restituiti da F3. Idempotente: chiamare F4 più volte con gli stessi ID non produce errori né duplicati.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body:

```
{
  "registration_ids": [42, 43, 44]  // Lista degli ID registrazione da marcare
  come sincronizzati – obbligatorio
}
```

Tabelle DB coinvolte:

licenses (aggiornamento `vendor_synced`)

Controlli:

- JWT vendor valido
- Tutti gli ID in `registration_ids` esistono e appartengono a questo vendor
- Se già confermati → no-op su quei record, risposta 200 comunque

Risposte:

```
// 200 - Conferma registrata
{
  "status": "confirmed",
  "confirmed_ids": [42, 43],
  "already_synced_ids": [44]  // IDs già sincronizzati in precedenza
  (idempotenza)
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 404 - Uno o più ID non trovati
{
  "error_code": "REGISTRATION_NOT_FOUND",
  "message": "Uno o più ID registrazione non trovati o non appartengono a questo
  vendor",
  "details": { "invalid_ids": [99] },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

Il produttore conferma di aver ricevuto e processato le iscrizioni. Il servizio marca le relative licenze con `vendor_synced = true`.

8.3 Attivazione licenza a pagamento

Quando un cliente acquista una licenza mensile o annuale, il produttore gestisce il pagamento sul proprio sistema e notifica il servizio tramite F5.

F5 – Attivazione licenza a pagamento

POST /api/vendor/license/activate

Specifica tecnica endpoint (rif. Endpoint v5.1)

Scopo:

Attiva una licenza a pagamento (mensile o annuale) per un cliente già registrato. Sostituisce il contratto trial o provvisorio con uno standard. Supporta anche la creazione di licenze provvisorie (is_provisional = true) in attesa di conferma pagamento. Idempotente tramite header Idempotency-Key: in caso di timeout o doppio invio, restituisce la risposta originale dalla cache (TTL 24h, tabella idempotency_keys).

Header:

```
Authorization: Bearer <jwt_vendor>
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000 // UUID univoco per
questa operazione – obbligatorio
```

Request body:

```
{
  "vat_number":      "IT12345678901",           // P.IVA del cliente –
obbligatorio
  "product_key":     "PK-XYZ",                  // Chiave prodotto – obbligatorio
  "license_type":    "monthly|annual",          // Tipo di licenza – obbligatorio
  "starts_at":       "2026-06-09T00:00:00Z",    // Data inizio licenza –
obbligatorio
  "expires_at":      "2026-07-09T23:59:59Z",    // Data scadenza – obbligatorio
  "max_users":       5,                         // Numero massimo utenti
  "modules":         ["modulo_a", "modulo_b"],   // Moduli abilitati – obbligatorio
  "is_provisional":  false                      // true = licenza provvisoria,
upgradabile tramite F5
}
```

Tabelle DB coinvolte:

clients (lettura), licenses (aggiornamento vecchia, inserimento nuova), idempotency_keys (lettura/inserimento)

Controlli:

- JWT vendor valido
- vat_number + product_key esistono nel DB con cliente attivo
- license_type è monthly o annual
- expires_at > starts_at
- Header Idempotency-Key: se già processato → risposta dalla cache con _cached: true

Risposte:

```
// 201 - Licenza attivata
{
  "status": "activated",
  "license_key": "lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h",
  "license_type": "monthly",
  "starts_at": "2026-06-09T00:00:00Z",
  "expires_at": "2026-07-09T23:59:59Z",
  "modules": ["modulo_a", "modulo_b"],
```

```

    "is_provisional": false
  }

// 200 - Risposta dalla cache (Idempotency-Key già elaborata)
{
  "status": "activated",
  "license_key": "lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h",
  "license_type": "monthly",
  "starts_at": "2026-06-09T00:00:00Z",
  "expires_at": "2026-07-09T23:59:59Z",
  "modules": ["modulo_a", "modulo_b"],
  "is_provisional": false,
  "_cached": true
}

// 400 - Date non valide
{
  "error_code": "INVALID_DATE_RANGE",
  "message": "expires_at deve essere successivo a starts_at",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 400 - Idempotency-Key mancante
{
  "error_code": "MISSING_IDEMPOTENCY_KEY",
  "message": "L'header Idempotency-Key è obbligatorio per questo endpoint",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 404 - Cliente non trovato
{
  "error_code": "CLIENT_NOT_FOUND",
  "message": "Nessun cliente trovato per vat_number e product_key forniti",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

```

Il produttore invia i dettagli della nuova licenza. Il servizio verifica l'esistenza del cliente, controlla la coerenza delle date, disattiva la licenza precedente (inclusa eventuale provvisoria), crea la nuova licenza attiva e **rigenera il file crittografato offline e la chiave di decrittazione**.

In questa fase vengono inviati i seguenti messaggi:

Email attivazione licenza → Cliente (event_code: LICENSE_ACTIVATED)

Oggetto: Licenza {product_name} attivata – Accesso confermato

Gentile {company_name},

siamo lieti di comunicarLe che la sua licenza di {product_name} è stata attivata con

successo.

Dettagli licenza:

Tipo: {license_type}

Valida fino al: {expires_at}

Può accedere al servizio da subito.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Messaggio in-app → Cliente (event_code: LICENSE_ACTIVATED)

Titolo: Licenza attivata

Testo: La licenza di {product_name} è stata attivata con successo ed è valida fino al {expires_at}. Può accedere a tutte le funzionalità incluse nel suo abbonamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

8.4 Avvisi di scadenza e disattivazione automatica

Job schedulati gestiscono le notifiche anticipate di scadenza. Gli avvisi sono **sospesi durante una Licenza Provvisoria attiva**, poiché il cliente ha già dimostrato l'intenzione di rinnovare.

Trial Demo: avvisi 7, 3 e 1 giorno prima della scadenza.

Licenza mensile: avvisi 7, 3 e 1 giorno prima della scadenza.

Licenza annuale: avvisi a 3 mesi, 2 mesi, 6 settimane, 1 mese, 3 settimane, 2 settimane, 10 giorni, 7 giorni, 3 e 1 giorno prima.

Per ogni avviso viene inviata un'email e un messaggio in-app al cliente e triggerato un GET ALARM con alarm_code = LICENSE_EXPIRING.

Messaggi inviati per ogni avviso di scadenza (il testo è lo stesso, cambia solo {days_remaining}):

Email scadenza imminente → Cliente — esempio Trial Demo (event_code: TRIAL_EXPIRING)

Oggetto: La tua Trial Demo di {product_name} scade tra {days_remaining} giorni

Gentile {company_name},

le ricordiamo che il periodo di prova di {product_name} si concluderà tra {days_remaining} giorni, il {expires_at}.

Per continuare ad usufruire del servizio, la invitiamo a procedere con l'acquisto di una licenza.

Al termine della Trial Demo, senza un abbonamento attivo, l'accesso verrà sospeso automaticamente.

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Email scadenza imminente → Cliente — esempio licenza mensile (event_code: MONTHLY_EXPIRING)

Oggetto: La tua licenza mensile di {product_name} scade tra {days_remaining} giorni

Gentile {company_name},

le ricordiamo che la sua licenza mensile scadrà tra {days_remaining} giorni, il {expires_at}.

Per garantire la continuità del servizio, procedere con il rinnovo prima della data di scadenza.

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Email scadenza imminente → Cliente — esempio licenza annuale (event_code: ANNUAL_EXPIRING)

Oggetto: La tua licenza annuale di {product_name} scade tra {days_remaining} giorni

Gentile {company_name},

le ricordiamo che la sua licenza annuale scadrà tra {days_remaining} giorni, il {expires_at}.

Per garantire la continuità del servizio, procedere con il rinnovo prima della data di scadenza.

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Messaggio in-app → Cliente (vale per tutti i tipi di licenza in scadenza)

Titolo: Licenza in scadenza (es. Trial Demo in scadenza / Licenza mensile in scadenza / Licenza annuale in scadenza)

Testo: La licenza di {product_name} scadrà tra {days_remaining} giorni, il {expires_at}. Per garantire la continuità del servizio è necessario procedere con il rinnovo o l'acquisto di un abbonamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Il giorno stesso della scadenza, se la licenza non è stata rinnovata, il servizio la disattiva automaticamente (status → expired) e triggera GET ALARM con alarm_code = LICENSE_EXPIRED.

Messaggi inviati il giorno della scadenza:

Email licenza scaduta → Cliente (event_code: LICENSE_EXPIRED)

Oggetto: Licenza {product_name} scaduta – Accesso sospeso

Gentile {company_name},

le comunichiamo che la sua licenza di {product_name} è scaduta in data {expires_at} e l'accesso al servizio è stato sospeso.

Per ripristinare l'accesso, la invitiamo a contattarci:

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Messaggio in-app → Cliente (event_code: LICENSE_EXPIRED)

Titolo: Licenza scaduta

Testo: La licenza di {product_name} è scaduta il {expires_at}. L'accesso al servizio è stato sospeso. Per ripristinare l'accesso è necessario procedere con il rinnovo o l'acquisto di una nuova licenza.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Email licenza scaduta → Produttore

Oggetto: Licenza scaduta – {company_name} – {product_name}

Gentile team,

si comunica che la licenza del seguente cliente è scaduta e l'accesso è stato sospeso.

Dettagli:

Azienda: {company_name}
Prodotto: {product_name}
Data scadenza: {expires_at}

Cordiali saluti,
Il sistema di gestione licenze

8.5 Licenza Provvisoria

La Licenza Provvisoria tutela sia il produttore che il cliente nel periodo tra l'emissione della fattura e la ricezione del pagamento. Evita che il cliente usufruisca di una licenza completa senza pagare, garantendo allo stesso tempo la continuità del servizio durante il normale iter amministrativo. Durante la licenza provvisoria tutti gli avvisi di scadenza vengono sospesi.

F7 – Emissione fattura verso cliente

POST /api/vendor/invoice/issued

Il produttore comunica di aver emesso una fattura verso un cliente, specificando la durata della licenza provvisoria (es. 30 giorni). Se la licenza corrente è in scadenza o già scaduta, il servizio attiva automaticamente una Licenza Provvisoria, **rigenera il file crittografato offline e la chiave di decrittazione**, sospende gli avvisi di scadenza e triggera GET ALARM con alarm_code = PROVISIONAL_ACTIVATED.

In questa fase vengono inviati i seguenti messaggi:

Email Licenza Provvisoria → Cliente (event_code: PROVISIONAL_ACTIVATED)

Oggetto: Licenza provvisoria attivata – {product_name}

Gentile {company_name},

in seguito all'emissione della fattura da parte del nostro team, le comunichiamo che è stata attivata una

Licenza Provvisoria per {product_name}, valida fino al {expires_at}.

Questo le garantisce la continuità del servizio nel periodo di elaborazione del pagamento.

Non riceverà ulteriori avvisi di scadenza durante questo periodo.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Messaggio in-app → Cliente (event_code: PROVISIONAL_ACTIVATED)

Titolo: Licenza provvisoria attivata

Testo: È stata attivata una Licenza Provvisoria per {product_name}, valida fino al {expires_at}. Il servizio è garantito durante il periodo di elaborazione del pagamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

F8 – Conferma pagamento ricevuto

POST /api/vendor/invoice/paid

Il produttore conferma la ricezione del pagamento. Il servizio disattiva la licenza provvisoria, crea la nuova licenza definitiva, **rigenera il file crittografato offline e la chiave di decrittazione**, riattiva gli avvisi di scadenza e triggera GET ALARM con alarm_code = PAYMENT_CONFIRMED.

In questa fase vengono inviati i seguenti messaggi:

Email attivazione licenza → Cliente (event_code: LICENSE_ACTIVATED)

Oggetto: Licenza {product_name} attivata – Accesso confermato

Gentile {company_name},

siamo lieti di comunicarle che la sua licenza di {product_name} è stata attivata con successo.

Dettagli licenza:

Tipo: {license_type}

Valida fino al: {expires_at}

Può accedere al servizio da subito.

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Messaggio in-app → Cliente (event_code: LICENSE_ACTIVATED)

Titolo: Licenza attivata

Testo: La licenza di {product_name} è stata attivata con successo ed è valida fino al {expires_at}. Può accedere a tutte le funzionalità incluse nel suo abbonamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Caso: pagamento non ricevuto entro la scadenza provvisoria. La licenza viene disattivata automaticamente senza avvisi anticipati. Viene generato solo un messaggio in-app al momento della disattivazione.

In questa fase viene generato solo il seguente messaggio in-app (nessuna email):

Messaggio in-app → Cliente (event_code: PROVISIONAL_EXPIRED)

Titolo: Licenza scaduta

Testo: La Licenza Provvisoria di {product_name} è scaduta il {expires_at}. L'accesso al servizio è stato sospeso. Per ripristinare l'accesso contattare il produttore.

Contattare {contact_email} oppure {contact_phone}.

8.6 Caso eccezionale: tentativo di ri-registrazione bloccato

La license_key viene emessa una sola volta per ogni coppia cliente+prodotto. Se un client tenta di registrarsi nuovamente con la stessa P.IVA e product_key, il servizio blocca il tentativo con errore 409 e genera un messaggio in-app che spiega come procedere.

In questa fase viene generato solo il seguente messaggio in-app (nessuna email):

Messaggio in-app → Cliente (event_code: REGISTRATION_BLOCKED)

Titolo: Registrazione non consentita

Testo: Non è possibile completare la registrazione. La Trial Demo per questo prodotto è già stata utilizzata in precedenza con questo account. Per accedere nuovamente al servizio è necessario contattare direttamente il produttore.

Contattare {contact_email} oppure {contact_phone}.

Questo messaggio viene inserito nella tabella messages con channel = in_app e delivered_at = NULL, in modo che venga restituito alla prossima chiamata C5 del client.

8.7 Monitoraggio attività client e notifica inattività

Ad ogni chiamata C5, il servizio aggiorna `last_seen_at` nella tabella `client_activity_logs`. Un job schedulato verifica periodicamente se ci sono client con licenza attiva che non effettuano chiamate C5 da almeno 7 giorni consecutivi. In tal caso invia un'email al produttore e aggiorna `inactivity_notified_at` per evitare invii duplicati.

In questa fase viene inviata la seguente email al produttore (nessun messaggio in-app):

Email client inattivo → Produttore (event_code: CLIENT_INACTIVE)

Oggetto: Attenzione: client inattivo da 7 giorni – {company_name} – {product_name}

Gentile team,

si comunica che il seguente cliente non risulta attivo sul servizio da almeno 7 giorni.
Il sistema non ha ricevuto chiamate dalla libreria client installata presso di loro.

Dettagli:

Azienda: {company_name}
Prodotto: {product_name}
Licenza: {license_type}
Scadenza licenza: {expires_at}
Ultima attività rilevata: {last_seen_at}

Si consiglia di verificare con il cliente se il servizio è ancora in uso o se si sono verificati problemi tecnici.

Cordiali saluti,
Il sistema di gestione licenze

8.8 Riapertura dell'applicazione da parte di un cliente già registrato

Quando un cliente già registrato riapre l'applicazione, la libreria client chiama nuovamente C1. Il servizio riconosce che `vat_number` + `product_key` sono già associati a una licenza attiva e restituisce direttamente i dati della licenza esistente (`license_key`, tipo, scadenza) senza avviare un nuovo processo di registrazione e senza generare messaggi in-app.

La libreria client utilizza poi la `license_key` e il meccanismo di token refresh (C6) per riottenere un JWT valido e riprendere il funzionamento ordinario (C4 e C5).

=====

Endpoint aggiuntivi (rif. Endpoint v5.1)

Endpoint presenti nel documento Endpoint v5.1 e non corrispondenti ad alcuna chiamata già descritta sopra. Inseriti qui per completezza.

C7 — POST /api/client/change-email (nuovo in v4)

Scopo:

Avvia il cambio dell'indirizzo email del cliente. Invia un codice OTP alla nuova email per la verifica. Il cambio viene completato solo dopo la verifica tramite C7b.

Header:

```
Authorization: Bearer <jwt>
X-License-Key: lk_...
```

Request body:

```
{
  "new_email": "nuova@email.com"    // Nuovo indirizzo email - obbligatorio, deve
  essere valido
}
```

Tabelle DB coinvolte:

clients (lettura), otp_codes (inserimento)

Controlli:

- JWT valido
- new_email in formato valido
- new_email diverso dall'email corrente del cliente
- new_email non già in uso da un altro cliente per lo stesso prodotto

Risposte:

```
// 200 - OTP inviato alla nuova email
{
  "status": "otp_sent",
  "message": "Codice OTP inviato al nuovo indirizzo email. Verificare con C7b."
}

// 400 - Email non valida
{
  "error_code": "INVALID_EMAIL_FORMAT",
  "message": "Il formato dell'email non è valido",
  "details": { "field": "new_email" },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 400 - Email uguale a quella corrente
{
  "error_code": "EMAIL_SAME_AS_CURRENT",
  "message": "Il nuovo indirizzo email è uguale a quello attuale",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

```
// 409 - Email già in uso
{
  "error_code": "EMAIL_ALREADY_IN_USE",
  "message": "L'indirizzo email è già associato a un altro cliente",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

C7b — POST /api/client/verify-email-change (nuovo in v4)

Scopo:

Verifica il codice OTP inviato alla nuova email (da C7) e completa il cambio email aggiornando il campo `contact_email` nel DB.

Header:

```
Authorization: Bearer <jwt>
X-License-Key: lk_...
```

Request body:

```
{
  "otp_code": "391847"    // Codice OTP a 6 cifre ricevuto sulla nuova email -
                           obbligatorio
}
```

Tabelle DB coinvolte:

`clients` (aggiornamento), `otp_codes` (lettura/eliminazione), `otp_attempts` (lettura/aggiornamento)

Controlli:

- JWT valido
- OTP valido, non scaduto, corrisponde all'hash SHA256 in `otp_codes`
- Max 3 tentativi falliti

Risposte:

```
// 200 - Email aggiornata con successo
{
  "status": "email_changed",
  "new_email": "nuova@email.com"
}

// 401 - OTP errato
{
  "error_code": "INVALID_OTP",
  "message": "Codice OTP non valido",
  "details": { "attempts_remaining": 1 },
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 401 - OTP scaduto
{
  "error_code": "OTP_EXPIRED",
  "message": "Il codice OTP è scaduto. Richiedere un nuovo cambio email tramite C7.",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 403 - Troppi tentativi falliti
{
  "error_code": "OTP_MAX_ATTEMPTS",
```

```
"message": "Troppi tentativi falliti. Riprovare tra 30 minuti.",
"details": { "retry_after_seconds": 1800 },
"timestamp": "2026-06-09T10:00:00Z",
"request_id": "uuid"
}
```

F7 — POST /api/vendor/client/billing

Scopo:

Salva i dati di fatturazione del cliente (raccolti dall'ERP al primo acquisto). Operazione upsert: se i dati esistono già vengono aggiornati. Idempotente per natura (upsert). L'IBAN è opzionale: necessario solo se il fornitore utilizza addebito diretto SEPA; altrimenti il pagamento è gestito interamente nell'ERP.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body:

```
{
  "vat_number":      "IT12345678901",      // P.IVA del cliente - obbligatorio
  "product_key":     "PK-XYZ",             // Prodotto di riferimento -
obbligatorio
  "pec_email":       "admin@acme.pec.it",   // PEC per fatturazione elettronica
- obbligatorio se IT
  "sdi_code":        "ABC1234",            // Codice destinatario SDI -
alternativo a PEC se IT
  "billing_address": "Via Roma 1",         // Indirizzo di fatturazione -
obbligatorio
  "billing_city":    "Milano",              // Città - obbligatorio
  "billing_zip":     "20121",              // CAP - obbligatorio
  "billing_country": "IT",                 // Paese ISO - obbligatorio
  "iban":            "IT60X0542811101000000123456" // IBAN - opzionale, solo se
pagamento SEPA
}
```

Tabelle DB coinvolte:

clients (lettura), client_billing (upsert)

Controlli:

- JWT vendor valido
- vat_number + product_key esistono nel DB
- Se billing_country = IT: almeno uno tra pec_email e sdi_code obbligatorio

Risposte:

```
// 200 - Dati di fatturazione salvati
{
  "status": "saved",
  "vat_number": "IT12345678901"
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 404 - Cliente non trovato
{
  "error_code": "CLIENT_NOT_FOUND",
```

```

    "message": "Nessun cliente trovato per vat_number e product_key",
    "timestamp": "2026-06-09T10:00:00Z",
    "request_id": "uuid"
  }

// 422 - Dati fatturazione non validi
{
  "error_code": "INVALID BILLING DATA",
  "message": "Per clienti italiani è obbligatorio pec_email oppure sdi_code",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

```

F8 — POST /api/vendor/license/revoke

Scopo:

Revoca la licenza attiva o provvisoria di un cliente (es. per mancato pagamento). Imposta status = revoked, invalida l'offline_token e mette in coda email + messaggio in-app di notifica al cliente. Idempotente: se la licenza è già revocata, la chiamata è un no-op e restituisce 200.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body:

```

{
  "vat_number": "IT12345678901",           // P.IVA del cliente - obbligatorio
  "product_key": "PK-XYZ",                 // Prodotto da revocare - obbligatorio
  "reason": "Mancato pagamento"           // Motivo revoca - opzionale, usato nel
  template email
}

```

Tabelle DB coinvolte:

clients (lettura), licenses (aggiornamento status), messages (inserimento notifica)

Controlli:

- JWT vendor valido
- vat_number + product_key esistono nel DB
- Se licenza già revocata → no-op, risposta 200

Risposte:

```

// 200 - Licenza revocata
{
  "status": "revoked",
  "license_key": "lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h",
  "revoked_at": "2026-06-09T12:00:00Z"
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

// 404 - Licenza non trovata
{
  "error_code": "LICENSE_NOT_FOUND",
  "message": "Nessuna licenza attiva trovata per vat_number e product_key",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}

```

```
}
```

F9 — POST /api/vendor/auth/rotate-key (nuovo in v4)

Scopo:

Genera una nuova API key per il vendor e revoca quella corrente. La vecchia chiave rimane valida per un grace period (es. 1h) per permettere la transizione senza downtime. Lo storico delle chiavi viene salvato in vendors.api_key_history.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body: nessuno

Tabelle DB coinvolte:

vendors (aggiornamento api_key_hash, api_key_revoked_at, api_key_history)

Controlli:

- JWT vendor valido

Risposte:

```
// 200 - Nuova API key generata
{
  "new_api_key": "vk_nuova_chiave_in_chiaro_mostratasoloora", // Mostrata una
  sola volta
  "old_key_valid_until": "2026-06-09T13:00:00Z" // Grace period: 1h
}

// 401 - JWT non valido
{
  "error_code": "INVALID_JWT",
  "message": "Token non valido o scaduto",
  "timestamp": "2026-06-09T10:00:00Z",
  "request_id": "uuid"
}
```

Appendice tecnica — Tabelle DB e convenzioni

Convenzioni generali

Autenticazione client: Authorization: Bearer <jwt> + X-License-Key: <license_key>

Autenticazione vendor: Authorization: Bearer <jwt_vendor>

Formato errori:

```
{
  "error_code": "SNAKE_CASE_COSTANTE",
  "message": "Descrizione leggibile",
  "details": { "field": "campo_opzionale" },
  "timestamp": "ISO8601",
  "request_id": "uuid"
}
```

Riepilogo tabelle DB — campi principali

vendors

Campo	Tipo	Descrizione
id	INT PK	ID vendor
name	VARCHAR(255)	Nome fornitore

api_key_hash	VARCHAR(255)	Hash bcrypt dell'API key corrente
api_key_revoked_at	TIMESTAMP	Data revoca (NULL se attiva)
api_key_history	JSON	Storico chiavi precedenti
erp_alarm_url	VARCHAR(500)	URL endpoint O1 dell'ERP
created_at	TIMESTAMP	

vendor_general_setup (nuova — sezione 12)

Campo	Tipo	Descrizione
id	INT PK DEFAULT 1	Sempre 1 — record unico
vendor_id	INT FK	Vendor di questa istanza
default_check_interval_hours	INT DEFAULT 24	Frequenza default job schedulati

vendor_event_config (nuova — sezione 12)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
event_code	VARCHAR(50)	Es. NEW_REGISTRATION
enabled	BOOLEAN DEFAULT true	ON = job attivo, OFF = job non registrato
check_interval_hours	INT NULL	NULL = usa default_check_interval_hours
settings_json	TEXT	Config specifica (soglie, max_retries...)
last_run_at	TIMESTAMP	Ultimo avvio job

products

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
product_key	VARCHAR(50) UNIQUE	Chiave da distribuire nella libreria client
product_name	VARCHAR(255)	
trial_duration_days	INT	
trial_max_users	INT	
license_check_frequency_days	INT	Frequenza check C4

clients

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
product_id	INT FK	
vat_number	VARCHAR(50)	
company_name	VARCHAR(255)	
country	VARCHAR(10)	Codice ISO

contact_email	VARCHAR(255)	
contact_phone	VARCHAR(50)	
referent_name	VARCHAR(255)	
status	ENUM(pending, active)	
last_c5_at	TIMESTAMP	Ultima chiamata C5 (per inattività)
inactivity_notified_at	TIMESTAMP	Ultima notifica CLIENT_INACTIVE

licenses

Campo	Tipo	Descrizione
id	INT PK	
client_id	INT FK	
license_key	VARCHAR(255) UNIQUE	Generata con HMAC-SHA256 + salt
license_type	ENUM(trial, normal)	
status	ENUM(active, expired, revoked)	
starts_at	TIMESTAMP	
expires_at	TIMESTAMP	
max_users	INT	
vendor_synced	BOOLEAN DEFAULT false	true dopo conferma O1 (gestito dal job)
offline_token	TEXT	Crittografato AES-256-GCM
offline_token_expires_at	TIMESTAMP	

alarm_logs

Campo	Tipo	Descrizione
id	INT PK	
license_id	INT FK	
alarm_code	VARCHAR(50)	NEW_REGISTRATION, LICENSE_EXPIRING, LICENSE_EXPIRED
success	BOOLEAN	
retry_count	INT DEFAULT 0	
last_retry_at	TIMESTAMP	
next_retry_at	TIMESTAMP	
max_retries	INT DEFAULT 3	
permanently_failed	BOOLEAN DEFAULT false	true dopo max_retries esauriti
created_at	TIMESTAMP	

event_sent_log (nuova — sezione anti-duplicati)

Campo	Tipo	Descrizione
id	INT PK	

vendor_id	INT FK	
event_code	VARCHAR(50)	Es. NEW_REGISTRATION, LICENSE_EXPIRING_7DAYS
license_id	INT FK (nullable)	NULL per event client-level
client_id	INT FK (nullable)	
sent_at	TIMESTAMP	Quando è stato inviato
status	ENUM(success, failed)	
retry_count	INT DEFAULT 0	
next_eligible_at	TIMESTAMP (nullable)	Quando può essere rieseguito

UNIQUE INDEX: (vendor_id, event_code, license_id, DATE(sent_at)) WHERE status='success'

api_logs (nuova — logging audit)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
endpoint	VARCHAR(100)	Es. /api/client/register, /api/vendor/license/activate
method	VARCHAR(10)	GET, POST, PUT, DELETE
log_level	ENUM(CRITICAL, DEBUG)	CRITICAL sempre, DEBUG se enabled
client_id	INT FK (nullable)	
license_id	INT FK (nullable)	
request_body	TEXT (JSON)	
response_status	INT	HTTP status code
response_body	TEXT (JSON)	
ip_address	VARCHAR(50)	Per tracking anomalie
timestamp	TIMESTAMP	

Log Level:

- CRITICAL (sempre): C2, F5, F8, F1, F9, C1 (registration)
- DEBUG (se enabled): C4, C5, C6, F3, F2

security_alerts (nuova — anomalie)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
client_id	INT FK	
alert_type	VARCHAR(50)	ip_change, cloned_key, brute_force, suspicious
severity	ENUM(low, medium, high, critical)	
ip_address	VARCHAR(50)	IP sorgente
details	JSON	Dettagli anomalia

created_at	TIMESTAMP	
resolved	BOOLEAN DEFAULT false	

email_templates (nuova — template parametrizzabili)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
event_code	VARCHAR(50)	WELCOME, LICENSE_ACTIVATED, LICENSE_EXPIRED, ecc.
language	VARCHAR(2)	it, en
channel	ENUM(email, in_app)	Canale messaggio
subject	VARCHAR(255) (nullable)	NULL per in_app
body	TEXT	Con placeholder {product_name}, {expires_at}, ecc.
enabled	BOOLEAN DEFAULT true	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

UNIQUE: (vendor_id, event_code, language, channel)

license_history (nuova — storicità licenze)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
client_id	INT FK	
old_license_id	INT FK (nullable)	Licenza precedente
new_license_id	INT FK	Licenza attuale
change_type	VARCHAR(50)	trial_start, trial_to_normal, renewal, upgrade, downgrade
description	VARCHAR(255)	Es. "Trial 30gg → Normal 365gg"
old_license_type	VARCHAR(50) (nullable)	trial, normal
new_license_type	VARCHAR(50)	normal
old_duration_days	INT (nullable)	30, 365, 730, ecc.
new_duration_days	INT	
old_expires_at	TIMESTAMP (nullable)	
new_expires_at	TIMESTAMP	
changed_at	TIMESTAMP	
changed_by	VARCHAR(50)	'system', 'vendor', 'admin'

INDEX: (vendor_id, client_id, changed_at)

sdi_pec_verification_log (opzionale — verifica AGID future)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
client_id	INT FK	
sdi_code	VARCHAR(6) (nullable)	
pec_email	VARCHAR(255) (nullable)	
verification_type	ENUM(sdi, pec)	Tipo verifica
agid_response	JSON (nullable)	Risposta AGID (futura)
is_valid	BOOLEAN	
verified_at	TIMESTAMP	
expires_at	TIMESTAMP	Cache 24h

vendor_general_setup (aggiornata)

Campo	Tipo	Descrizione
id	INT PK DEFAULT 1	Sempre 1 — record unico
vendor_id	INT FK	Vendor di questa istanza
default_check_interval_hours	INT DEFAULT 24	Frequenza default job
api_logging_enabled	BOOLEAN DEFAULT true	Abilita log DEBUG
email_enabled	BOOLEAN DEFAULT true	Abilita invio email
email_provider	VARCHAR(50)	sendgrid, mailgun, brevo, smtp
email_from	VARCHAR(255)	Mittente email
email_reply_to	VARCHAR(255) (nullable)	Reply-to
email_signature_text	TEXT (nullable)	Firma email

vendor_event_config (aggiornata)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
event_code	VARCHAR(50)	NEW_REGISTRATION, LICENSE_EXPIRING, ecc.
enabled	BOOLEAN DEFAULT true	ON = job attivo
check_interval_hours	INT NULL	NULL = usa default
email_template_id	INT FK (nullable)	Link a email_templates
send_to_client	BOOLEAN DEFAULT true	Invia email cliente?
send_to_vendor	BOOLEAN DEFAULT false	Invia email vendor?

settings_json	TEXT	Config specifica (soglie, max_retries)
last_run_at	TIMESTAMP (nullable)	Ultimo avvio job

clients (aggiornata)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
product_id	INT FK	
vat_number	VARCHAR(30)	P.IVA / VAT
company_name	VARCHAR(255)	Ragione sociale
country	VARCHAR(2)	Codice ISO
contact_email	VARCHAR(255)	Email notifiche
contact_phone	VARCHAR(30)	Opzionale
referent_name	VARCHAR(255)	Opzionale
status	ENUM(pending, active)	
last_check_at	TIMESTAMP (nullable)	Ultima C4 (se frequency_hours)
last_c5_at	TIMESTAMP (nullable)	Ultima C5 (inattività)
inactivity_notified_at	TIMESTAMP (nullable)	Ultima notifica CLIENT_INACTIVE

client_billing (aggiornata)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
client_id	INT FK	
vat_number	VARCHAR(30)	
product_key	VARCHAR(50)	
sede_legale	VARCHAR(500)	OBBLIGATORIO — Sede legale
pec_email	VARCHAR(255) (nullable)	PEC (se IT)
sdi_code	VARCHAR(6) (nullable)	SDI (se IT)
billing_address	VARCHAR(500) (nullable)	Indirizzo OPZIONALE
billing_city	VARCHAR(100) (nullable)	Città OPZIONALE
billing_zip	VARCHAR(20) (nullable)	CAP OPZIONALE
billing_country	VARCHAR(2) (nullable)	Paese OPZIONALE
iban	VARCHAR(50) (nullable)	IBAN (opzionale)

document_number	VARCHAR(100) (nullable)	Numero documento
document_date	TIMESTAMP (nullable)	Data documento